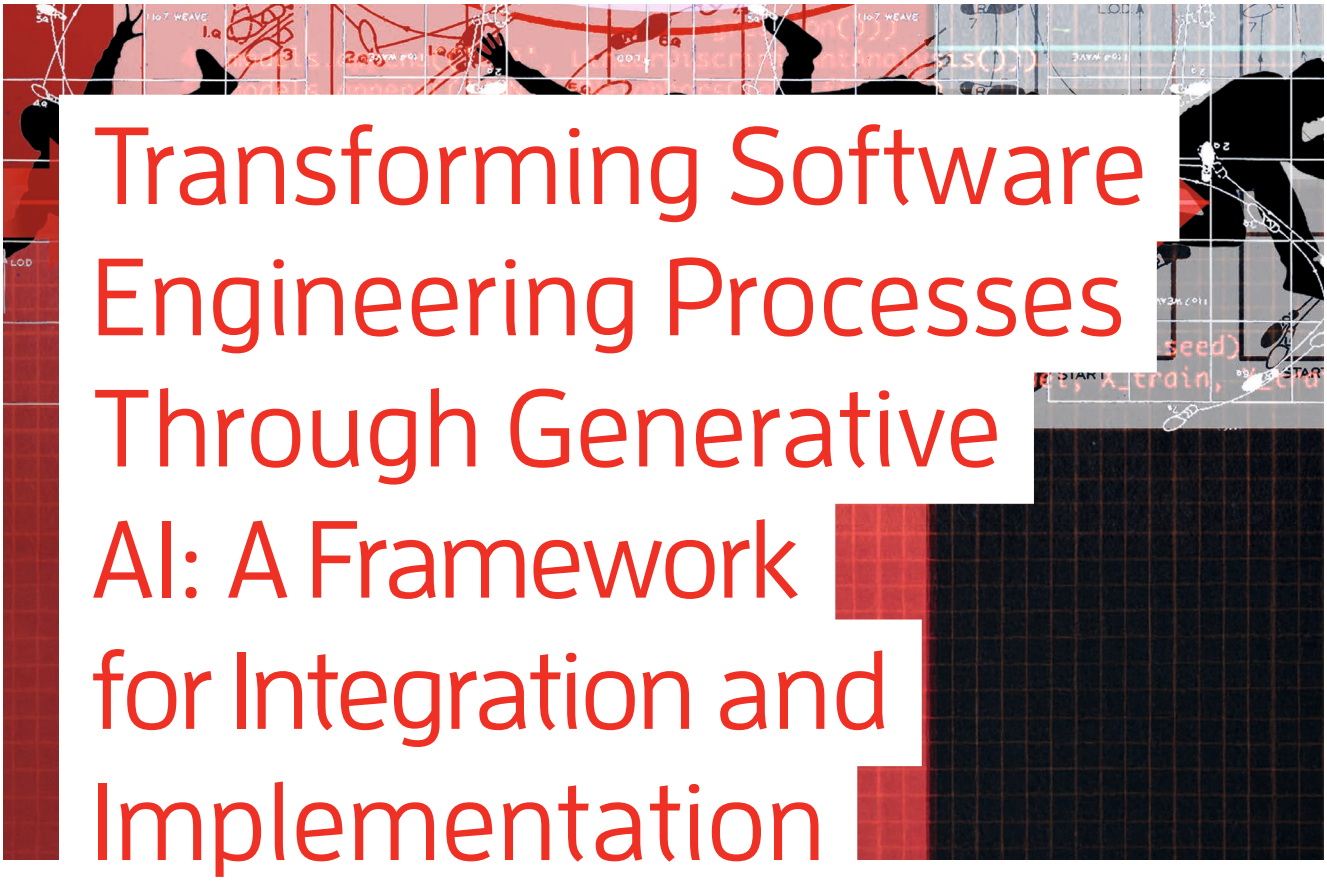

Transforming Software Engineering Processes Through Generative AI: A Framework for Integration and Implementation

Aybüke Yalçıner, TÜBİTAK, BİLGEM, Software Technologies Research Institute
Ebru Gökalp, Hacettepe University
Ahmet Dikici, ASELSAN, Inc.

In software engineering, integrating artificial intelligence (AI) technologies cuts costs and speeds up project timelines while improving workflow efficiency and software quality. We propose a framework that highlights tasks where leveraging generative AI can offer benefits through an analysis of current industry trends and research findings.

Generative artificial intelligence (AI) plays a role in the field of AI, focusing on generating unique content in different forms like text and visual media materials by using sophisticated models like Generative Pre-Trained Transformers (GPTs), Bidirectional Encoder Representations from Transformers (BERTs), generative adversarial networks (GANs), and large language models (LLMs). These models can handle tasks beyond data processing operations. For example, they can write poetry, tell stories, create pieces of code, design beautiful art, and have conversations that seem real enough. They're good at it because they've been taught using a lot of different information and examples to understand patterns and language rules well. As a result, they can sometimes produce outputs that resemble human work.

Digital Object Identifier 10.1109/MC.2025.3539347
Date of current version: 27 June 2025

The evidence gathered from real-world experiences strongly confirms the influence of generative AI in the realm of software development. GitHub Copilot[1] stands out as an example. A generative AI tool can successfully help in coding by suggesting code snippets and answering coding queries efficiently during the development process. The efficiency gains are significant: tasks are completed 55% faster with Copilot in practice. According to research conducted with approximately 450 developers, Accenture found that using Copilot boosts team output and enhances the quality of work delivered.[a] The positive impact extends to developer productivity well. Mercedes Benz study[b] noted an increase of 30 min a day through leveraging this tool. Comparisons with traditional estimation methods reveal that GPT2SP[2] can deliver 34%–57% more accurate story point estimations for tasks within the same project. It is also likely to boast an impressive accuracy improvement of 39%–49% for multiproject estimations. Generative AI is now changing the face of software development by increasing productivity, providing correctness, and automating activities such as code reviews and security scans. It maintains the focus of developers and ensures early detection of quality and security. Moreover, it massively reduces time for bug fixes and test creation. This game-changing technology reduces the time required for bug fixes postlaunch by 50% through automation during test procedure creation. In addition, it enhances development speed: developers can write code 35%–45% faster and refactor it 20%–30% faster. Eliminating repetitive work from developers' activities results in more creative, meaningful work. In this regard, 87% of developers with access to generative AI can focus on engaging work, compared to 50% without it. This increases team creativity and encourages experimentation to provide a better user experience.[3]

The benefits of generative AI have attracted practitioners' and academics' attention. The current landscape of software engineering (SE) shows increasing integration of LLMs, particularly generative AI, which is one of the most emerging tools in SE. Very recently, Rajbhoj et al.[4] identified successful applications in code generation, requirements gathering, and testing activities using such models. State-of-the-art studies show that LLMs speed up development processes and boost productivity while creating high-quality software artifacts.[5] However, problems of quality, trustworthiness, and ethical consequences that result from AI-generated solutions have been identified as major challenges.[6] Empirical studies reveal that generative AI is more deployed as an assistant collaborator than as a full automator, with effective interactions influenced by user context, among other factors, and organizational policies. In Khojah et al.,[7] despite its potential, further research is needed to address scalability, integration with existing development practices, and the long-term impact of generative AI on software quality and team dynamics. Despite the growing body of research, there remains a significant gap in our understanding of how to integrate generative AI across SE processes comprehensively. To our knowledge, there is currently no study in the literature that outlines the areas where generative AI can be applied in SE processes, discusses the challenges involved, presents a road map for successful implementation, and explains the potential future impacts and areas influenced by the widespread use of generative AI in SE.

This study aims to fill this gap by conducting a rigorous analysis to identify specific tasks within the SE processes defined in SWEBOK v4[8] that can significantly benefit from the application of generative AI technologies.

This analysis is intended to provide SE professionals with actionable insights for effectively utilizing generative AI. It also aims to highlight the inherent limitations of the technology while exploring the promising possibilities it offers for future advancements.

The remainder of this article is organized as follows. The next section explores specific applications of generative AI in each SE process, providing detailed implementation examples. This is followed by a comprehensive road map for integrating generative AI into existing software development workflows, along with a discussion of potential challenges and practical solutions for organizations that adopt these technologies. Finally, emerging trends and future implications for the SE field are presented, concluding with a summary of the research findings.

INTEGRATION OF GENERATIVE AI

The developed framework, given in Table 1, was created by analyzing and synthesizing the findings of two significant systematic literature reviews[9,10] conducted in 2024, one of which was conducted by the authors themselves.[9]

[a]https://www.youtube.com/watch?v=AAT4zCfzsHI.
[b]https://www.youtube.com/watch?v=HdAqqNM1i9c.

TABLE 1. Application of generative AI in SE processes.

SE process	Tools	Task	Explanation	Advantages of utilization of generative AI
Software requirements	ChatGPT, Claude AI, Requirements Assistant AI, IBM DOORS, and Google Bard	Anaphoric ambiguity treatment	Resolving ambiguities in natural language requirements to ensure clear interpretation	Enhancing clarity and precision by reducing interpretational uncertainties
		Requirements analysis and evaluation	Collecting, understanding, and documenting stakeholder needs and constraints	Accelerating requirements collection and enhancing documentation consistency
		Co-reference detection	Handling terminology variations across stakeholder documents over time	Eliminating inconsistencies and improving clarity and traceability
		Specification formalization	Translating requirements into precise, unambiguous formal language	Enhancing documentation precision, consistency, and interteam communication
		Use-case generation	Creating scenarios of system-user interactions	Improving system behavior understanding among developers and stakeholders
		Requirements classification	Categorizing requirements into functional and nonfunctional types	Developing a more effective software design road map
		Specification generation	Detailing requirement specifications	Ensuring correct solution development and facilitating stakeholder communication
		Requirements elicitation	Identifying software requirements	Enhancing understanding of software needs and reducing project failure risk
		Software specification synthesis	Automatically extracting requirements from natural language sources	Achieving high automation success, improving process efficiency and speed
Software architecture	ChatGPT, Claude AI, ArchiMate AI, Enterprise Architect AI, and Draw.io AI	System design	Creation of system architectures and components	Faster design iteration and reduced human error
Software design	ChatGPT, Claude AI, Figma AI, Sketch AI, Axure AI, and Zeplin AI	GUI retrieval	Finding and retrieving GUI designs or interfaces	Integration of text and visual information, and time and cost efficiency
		User design recommendation	Suggesting optimal user interface (UI) design	Improved user experiences and alignment with user needs
Software construction	ChatGPT, Claude AI, GitHub Copilot, TabNine, Sourcery AI, and SonarLint	Code representation	Encoding code semantics into distributed vector representations	Supporting downstream tasks like code search and generation
		Method name generation	Suggesting accurate method names	Improved code readability and understandability
		API documentation smells	Tracking API documentation quality	Enhanced developer experience
		Data analysis	Analyzing large datasets	Faster and more accurate analysis results
		Control flow graph generation	Illustrating program behavior	Supporting downstream tasks like code search and generation
		Instruction generation	Creating documentation from codebase, design, and requirements	Ensuring consistency, saving time, and improving accuracy

(*Continued*)

TABLE 1. (*Continued.*) Application of generative AI in SE processes.

SE process	Tools	Task	Explanation	Advantages of utilization of generative AI
		Code search	Extracting source code from large codebase	Enhanced code search and retrieval
		Code understanding	Deep analysis of code structure	Improved code maintenance and optimization
		Identifier normalization	Normalizing vocabulary in code snippets	Improving understanding and automation
		Type inference	Determining data types in code	Improving readability, simplifying maintenance, and minimizing runtime errors
		Code generation	Generating code snippets	Enhancing code-writing efficiency and accuracy
Software testing	ChatGPT, Claude AI, DeepCode, Selenium AI, Testim.io, and SonarQube AI	Vulnerability detection	Identifying system vulnerabilities	Improved detection accuracy and uncovering unknown vulnerabilities
		Testing automation	Assessing software application accuracy and reliability	Improved test coverage and reduced manual effort
		Defect detection	Identifying software defects	Enhanced bug detection and improved software reliability
		Static analysis	Analyzing source code	Improved accuracy and vulnerability identification
		Compiler fuzzing	Finding compiler vulnerabilities	Generating smart inputs and identifying deep compiler bugs
		Invariant prediction	Identifying critical consistent conditions	Enhanced testing and continuous improvement
		Test generation	Creating test cases	Identify test coverage gaps to ensure thorough testing
		Verification	Validating software correctness	Improved software reliability and security
		Fault localization	Identifying fault-triggering test cases	Increased probability of finding fault-inducing scenarios
		GUI testing	Verifying user interface (UI)	Faster, more comprehensive testing and improved UI consistency
		Decompilation	Translating compiled code to higher-level languages	Enhanced code understanding and faster reverse engineering
		Malicious code localization	Analyzing security-threatening code segments	Faster localization and reduced false positives
		Resource leak detection	Managing improper resource release	Optimized resource utilization and improved system performance
		Regression test prioritization	Organizing test-case execution order	Faster fault detection and efficient resource utilization
		Root-cause analysis	Identifying error origins	Improved problem solving and preventing error recurrence

(*Continued*)

TABLE 1. (*Continued*.) Application of generative AI in SE processes.

SE process	Tools	Task	Explanation	Advantages of utilization of generative AI
Software maintenance	ChatGPT, Claude AI, Snyk AI, Embold, Loggly AI, and Dynatrace	Story point estimation	Predicting task time and resources	Better task complexity assessment
		Software tool configuration	Optimizing tools for project needs	Reduced setup time and minimized errors
		Requirement implementation likelihood	Predicting requirement implementation probability	Improved resource allocation
		Cost estimation	Predicting required budget	Better budget control
		Delay estimation	Predicting potential project delays	Early bottleneck identification
		Code clone detection	Detects identical code snippets	Helps to improve code quality
		Debugging	Finding errors or defects in the software	Improves the accuracy and sampling efficiency of code generation and improves developer productivity
		Review/commit/code classification	Categorizes code changes and commits	Improves code-review efficiency and ensures high-quality, consistent code across the development process
		Logging	Systematic reporting of errors, info, warnings, and so on, and software application	Helps to identify and diagnose bugs and performance bottlenecks
		Code revision	Improvement of existing code	Helps with error detection and performance optimization, ensuring consistency
		Code coverage prediction	Identification of gaps in test coverage	Helps to improve test prioritization and enable efficient resource allocation and early defect detection
		Code-review defects repair	Identification of issues or defects found during code reviews	Speeds up code review, improves consistency, and helps to identify security risks, logic errors, and performance issues
		Docker file repair	Identification and resolution of issues in Docker file	Helps to improve the efficiency, security, and maintainability of Docker images
		Program merge conflicts repair	Identifies the issues that arise when integrating individual code changes	Resolves conflicts in a more efficient manner
		Tag recommendation	Recommends tags for software posts O&A sites	Better content organization and a more efficient, user-friendly platform
		Type error repair	Correction and detection of type error issues in code	Helps developers to resolve type errors quickly; more reliable software and improves developer productivity
		Maintainability of software likelihood	Predicting the long-term ease of maintaining and updating a software system as it evolves over time	Results in more efficient development processes over the long term, lowers maintenance costs, and enhances overall software quality

(*Continued*)

TABLE 1. (*Continued.*) Application of generative AI in SE processes.

SE process	Tools	Task	Explanation	Advantages of utilization of generative AI
SE management	ChatGPT, Claude AI, Monday.com AI, Jira AI, Trello AI, and Smartsheet AI	Story point estimation & Effort estimation	Prediction of time, resources and manpower to complete a specific task.	Better assessment of task complexity, improves project planning, and better decision making
		Software tool configuration	Software tools optimized for use according to project needs	Reduces setup time, minimizes human error, ensures that tools are consistently configured, and enhances overall productivity
		Requirement implementation likelihood	Prediction of requirement implementation probability	Helps teams to prioritize high-likelihood requirements, optimize resource allocation, and mitigate risks, leading to more successful project outcomes
		Cost estimation	Prediction of required budget to complete a specific task	Budget control, improves project planning.
		Delay estimation	Prediction of potential delays in a software development project	Identifying bottlenecks and risks early and ensures better project management
		Document changes prediction	Prediction of potential changes in the project documents	Reducing the risk of outdated or inconsistent information
		Delivery capability estimation	Prediction of a teams' ability to deliver a project within the given constraints	Helps teams to set realistic expectations, improve planning, and ensure on-time delivery

Insights from these reviews were used to systematically map generative AI applications across seven core SE processes defined in SWEBOK v4.[8] Although these reviews offer valuable insights into the applications of generative AI in SE, there still exists a notable gap in our understanding of its comprehensive integration throughout the software development lifecycle. Insights from these reviews were examined and systematically mapped to generative AI applications across seven core SE processes defined in SWEBOK v4.[8] The given study, while recognizing the importance of the other six knowledge areas described in the SWEBOK v4,[8] focuses primarily on the following seven. The reason for this focus is that a narrower scope allows for deeper and more actionable insights into highly relevant domains in the current application of generative AI technologies. Additionally, we aim to provide empirical evidence where the measurable impacts can be easily assessed, lay the groundwork for future research in the other knowledge areas, and highlight fields with high potential for industry adoption shortly. This mapping was conducted by three authors, each with more than 10 years of experience in academia and industry, working independently. Interrater reliability was assessed by using Cohen's kappa coefficient (κ=0.85), which indicates strong agreement. Any discrepancies were resolved through structured discussion sessions.

The framework identifies seven SE processes that are particularly conducive to generative AI integration: software requirements, software architecture, software design, software construction, software testing, software maintenance, and SE management, as illustrated in Figure 1. Table 1 provides a detailed overview of the specific tasks within each process where generative AI can provide measurable benefits. It also outlines the potential advantages of leveraging generative AI for these tasks and recommends appropriate AI tools for each SE process, ensuring that teams can enhance their efficiency and effectiveness.

Software requirements

The success or failure of SE projects depends on accurately gathering and interpreting the requirements. Generative AI is a powerful tool for addressing these challenges by simplifying ambiguous language, resolving semantic ambiguities, creating detailed user-centered use cases, improving stakeholder communications, and aligning client expectations with the delivered project outcomes.

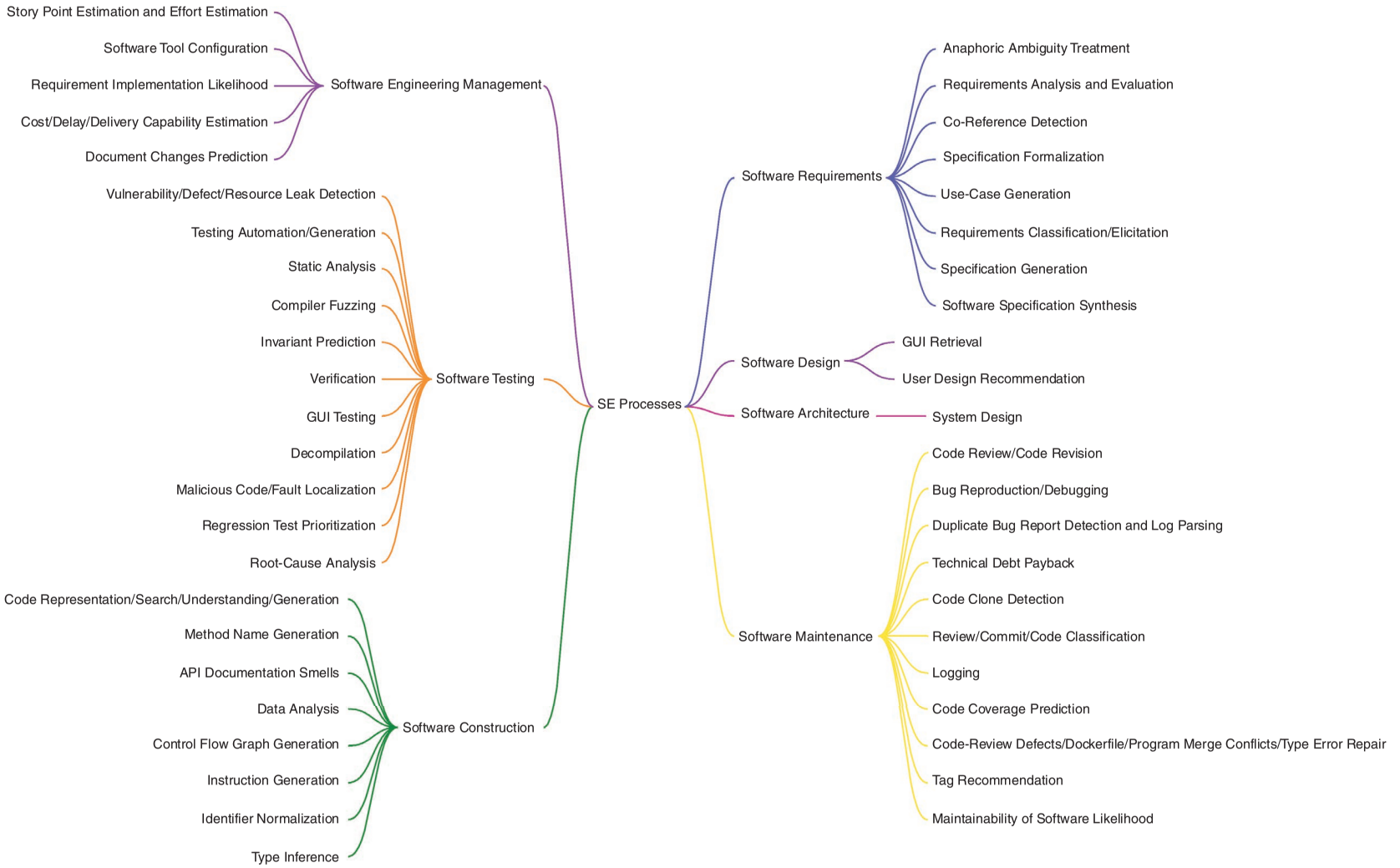

FIGURE 1. The proposed framework for utilization of generative AI in SE processes. API: application programming interface.

Software architecture

Architectural diagrams are necessary for reviewing and enhancing the long-term sustainability of a software system. This is enhanced with generative AI, where high-level design prototypes are built, analyzing existent codebases for ideal architectural patterns, identifying early vulnerabilities, and recommending best practices based on historical data. Ultimately, this improves the efficiency and reliability of software architecture.

Software design

In the software design phase, generative AI uses several machine learning methods, such as LLMs with natural language processing, to create detailed design models that help to optimize design decisions and improve the overall design process for functional and nonfunctional system requirements more accurately and quickly.

Software construction

Code quality is essential for the success of other phases in the software lifecycle. Well-constructed, efficient, and readable code simplifies testing and maintenance, especially during changes in project requirements, making refactoring easier and less error-prone. Although much has been written about coding best practices, few developers consistently apply these principles when writing code. Generative AI transforms development by automating the creation of high-quality code snippets, providing real-time optimization suggestions, identifying potential bugs before implementation, streamlining debugging processes, and ensuring consistent coding standards with greater agility.

STATE-OF-THE-ART STUDIES SHOW THAT LLMs SPEED UP DEVELOPMENT PROCESSES AND BOOST PRODUCTIVITY WHILE CREATING HIGH-QUALITY SOFTWARE ARTIFACTS.

Software testing

Testing is an important quality control mechanism that defines performance benchmarks and reduces potential issues. This is how generative AI will completely change testing: by automatically generating test cases, intelligently identifying bugs, anticipating error conditions, continuously learning from historical test results, and reducing manual testing efforts. Generative AI continuously improves testing techniques, raising software reliability and user satisfaction to higher levels.

Software maintenance

Maintenance is a resource-intensive yet very crucial software development lifecycle phase. Generative AI improves the process by detecting code inefficiencies, recommending targeted improvements, automating dependency updates, identifying potential security vulnerabilities, and reorganizing code. Proactive generative AI algorithms provide suggestions that turn maintenance into a proactive activity rather than a reactive one.

SE management

Effective SE management enables the delivery of projects on time, and within budget, while maintaining customer satisfaction and ensuring high-quality deliverables; it requires sophisticated resource coordination and predictive abilities. Generative AI will significantly enhance such practices by providing unparalleled management insights through better resource allocation, efficient usage of resources, proactive risk prediction, intelligent project scheduling, and accurate cost and effort estimation. It leverages insights from past project performance to enhance budgeting accuracy and enables real-time progress monitoring to predict on-time delivery. Generative AI combines historical project data with advanced predictive analytics, enabling managers to make data-driven decisions that improve the rate of project success.

Generative AI is transforming different aspects of SE, including agile software development,[11] by significantly enhancing how teams work and collaborate.[12] Mahboob et al.[13] discuss the role of generative AI in different agile software development approaches, such as XP or Lean, and agile practices, such as pair programming, test-first development, continuous integration, and refactoring, to enhance the productivity of agile teams. This can be highly beneficial for Scrum teams, as generative AI serves as a valuable assistant for key roles. For Scrum masters, this provides strategic event-facilitation

suggestions, presents insightful performance metrics, and detects bottlenecks in the workflow effectively. Similarly, for a product owner, this streamlines backlog prioritization, refines user stories, assists in market analysis, and offers predictive analytics.[11] Especially generative AI is effective in supporting agile practices like pair programming, test-driven development, continuous integration, and refactoring. Many studies have shown that the integration of generative AI into Scrum processes leads to significant improvements in team efficiency, innovation potential, and the quality of decision making, along with accelerating software delivery timelines. Embracing this technology will facilitate agile teams to get to higher levels of effectiveness and success that could not have been reached before.

IMPLEMENTATION OF GENERATIVE AI

Major integrational challenges of generative AI into SE processes, among others, include issues on integrations to current SE tools and workflows, high model complexities, lack of interpretability, very costly computations and implementations, ethics concerns, and responsibility for the inclusion of AI. The most particularly felt complexities of the models do include an algorithmic burden when it comes to special operations of the AI in given tasks considered during SE.[14] Despite these challenges, the benefits of adopting generative AI into SE processes are significant. Software organizations are eager to adopt it, but careful planning and execution are crucial to fully realize its long-term benefits. Otherwise, a poorly managed implementation can result in complex and unmanageable systems. To ensure success, it is essential to follow a structured road map and focus on the key areas that support AI adoption and effective data management.[15] The implementation steps are illustrated in Figure 2.

Initial assessment and goal definition

The first phase focuses on assessment and goal setting. Organizations must evaluate potential high-impact areas including requirements engineering, code generation, testing, and security, while defining clear objectives and success metrics for their implementation.

Determine how to manage data

The second phase is vital for establishing a robust data management framework, which is critical for the success of any organization. The development of holistic data storage and access protocols makes sure that our data are secure and of high quality. This captures accurate depictions of real-world scenarios and further refines the relevance and efficacy of generative AI models. Organizations must now implement metadata tracking and governance systems while enabling for capabilities of real-time data integration. A well-considered data management strategy forms the backbone of successful generative AI adoption. Training of generative AI models requires high-quality, accurate, and relevant data to ensure the optimal

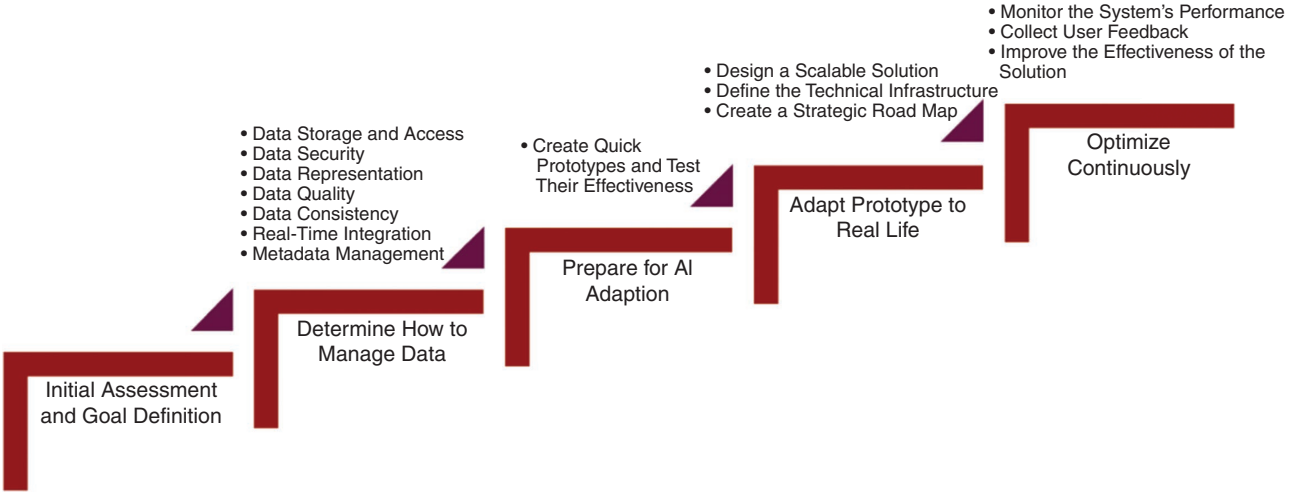

FIGURE 2. The steps to be followed for implementing generative AI.

results. This mainly concerns the determination of how and where the data will be stored and who will have access to them. Stringent data security should be implemented to ensure that sensitive information does not leak into the hands of unauthorized persons. It is also important to balance data to reflect real-life situations to increase the reliability of the model significantly. Implementing rigorous processes for data quality and consistency is crucial to maintaining accuracy across systems. Real-time data integration will make our AI models dynamic and responsive, thus best positioned to adapt to changing needs. Finally, effective metadata management enables lineage tracking and enhances governance for accountability and traceability. If all these critical areas are addressed, organizations are in a better position to provide a strong foundational framework for generative AI, thereby helping to ensure the fantastic performance of SE processes.

Prepare for AI adaption

Tool selection and prototyping is an important third phase, taking from two to eight weeks. At this stage, organizations make a strategic selection of the appropriate AI tools like GPT-4, Codex, or Copilot for organizational need analysis. Teams design rapid prototypes for the most important use cases, ensuring that these are in line with larger business objectives. The resources required are also identified in the process, while test applications are tried out under real-world conditions. Moreover, it is essential to assess the data and resources that are required for each use case as well as any gaps or technical challenges. This prototyping allows for extensive testing of the practical applications of AI and also forms an important step in the validation of concepts before full-scale implementation, therefore laying the grounds for success.

Adapt prototype to real life

Production implementation is the fourth phase and takes two to four months. During the first two to four weeks, organizations develop scalable infrastructure and devise an implementation road map. At this stage, full deployment across business units occurs as the prototype goes live in a production system. This includes extensive testing of all components of the system, coupled with intensive team training. To perfect the prototype into a fully operational generative AI system, precise fine-tuning ensures its reliability and effectiveness. Additionally, internal teams should receive proper training to ensure smooth assimilation and optimal usage of the new system. This concludes with the capability to make necessary changes and achieve successful large-scale implementation, laying a foundation for organizational success in general.

Optimize continuously

Continuous optimization is key in the final stage to ensure that the system remains effective and efficient over time. An organization should continuously monitor performance metrics, elicit user feedback actively, and scale data pipelines accordingly when required. Organizations can effectively meet emerging demands by enhancing system integration and strengthening security measures.

At this stage, all the modifications in the data pipeline must adapt to handle the increase in the volume of information. Moreover, integrating with other business systems and processes allows organizations to introduce new functionality in response to user needs and feedback. In addition, improving security protocols is essential for protecting sensitive data; it is also important to develop tools for continuous performance monitoring and optimization. Ethical considerations need to be addressed. Organizations have to be sure that their operation complies with emerging regulations and involves responsible behavior in practice. Building ethical guidance into the use of generative AI concerning data privacy, decision making, and security will enhance integrity while building trust. As companies need to comply with evolving regulatory frameworks, generative AI models must be transparent, explainable, and free from biases. As generative AI continues to surge into prominence, a commitment to organizational readiness for adoption and compliance will be necessary to realize long-term success and enable responsible deployment.

CHALLENGES OF GENERATIVE AI

Although this advanced technology can greatly speed up and enhance the development of solutions, which increases efficiency overall, it has to be engaged quite cautiously, given the presence of prominent risks. Organizations need to intelligently work their way around any mishaps that can lead to compromising project outcomes. These challenges include, as listed in Table 2, professional and ethical issues beyond technical aspects. Addressing them effectively demands strategic and sensitive stewardship so as not to incur unforeseen adverse

impacts. To realize the full power of generative AI, organizations should engage in a holistic approach to balance between technological innovation and human expertise. It means appropriate framing for responsible AI adoption, ensuring that such powerful tools augment and do not replace human intelligence, that is, developing adaptive implementation strategies with the key premises of transparency, continuous learning, and ethical considerations. Positioning generative AI more as a collaborative partner and less as a pure substitute for human developers allows organizations to realize its transformational potential while mitigating many of the associated risks. The dynamic and thought-provoking presentation needs to place generative AI in its proper context: it is no more than an advanced tool designed to augment rather than replace human creativity and problem-solving skills.[16]

FUTURE TRENDS

The future of generative AI in SE represents a significant transformation that could impact numerous aspects of the field.[17,18] We foresee a radical shift from traditional, experience-based approaches to more sophisticated data-driven decision-making processes as generative AI becomes even more pervasive within development workflows. This transition makes basic activities like design, testing, and deployment faster and more accurate, which results in faster and more dependable software delivery.

Generative AI in the field of DevSecOps will play an important role in security by offering early vulnerability detection and remediation across the development lifecycle. This proactive approach to risk mitigation will lead to higher application resilience. In addition, software development platforms are becoming easier and more intuitive to use, and development is possible with little to no formal coding knowledge among users. Technology democratization will become a key trend, with software development platforms becoming more accessible. Nontraditional coders will be better equipped to build apps, extending the innovation lifecycle and diversifying the talent pool. It should broaden access and eliminate traditional barriers to entry in software development. The role of cloud-based solutions will continue to increase, given their highly scalable and flexible nature. As the maturity of cloud computing advances, organizations will be more agile and quick to adapt to market dynamics and technological shifts. The convergence of cloud technologies with generative AI will make software infrastructures more responsive and intelligent. Quantum computing represents another frontier that will challenge current programming paradigms, requiring new approaches to software development. Developers will need to continuously evolve their skills, embracing new tools and techniques that leverage quantum computational capabilities. This technological disruption will push the boundaries of computational limits. Ethical considerations and evolving regulatory frameworks will increasingly shape technological development. New programming languages and development practices will emerge, prioritizing performance, simplicity, and robust security features. These advancements will not only redefine SE but also create diverse career opportunities for emerging talent.

TABLE 2. Challenges of generative AI.

Challenge	Explanation
Risks of overreliance on AI-generated code	Developers may assume that AI-generated code is inherently reliable, but such assumptions can lead to security vulnerabilities and errors, compromising software quality and project integrity.
Data privacy and security risks in AI training	Training AI systems requires large datasets, which may inadvertently expose sensitive project data, creating significant risks to confidentiality and security.
Financial implications of AI system development	The development and implementation of generative AI models can be resource intensive, involving high costs for model optimization, training, and efficient data handling.
Bias and adaptability challenges in AI models	Poorly balanced training datasets result in biased AI outputs, and with the evolution of SE, AI models have to be constantly updated to keep them effective, which adds complexity and costs.
Impact on developer creativity and problem solving	Overreliance on generative AI can lead to the loss of creativity and critical thinking among developers because they could stop developing original solutions and just stick with what the AI model suggests.
Concerns over job security in the age of AI	The widespread adoption of generative AI raises a wide range of concerns among developers regarding job security as AI would potentially replace human roles and make the profession yet uncertain.

ABOUT THE AUTHORS

AYBÜKE YALÇINER is a senior researcher at the Software Technologies Research Institute (YTE), The Informatics and Information Security Research Center (BILGEM), Scientific and Technological Research Council of Turkey (TUBITAK), Ankara 06530, Turkey. Her research interest includes data driven software engineering. Yalçıner received her master's degree in computer science from Hacettepe University. Contact her at aybukeyalciner@hacettepe.edu.tr.

EBRU GÖKALP is an associate professor in the Computer Engineering department at Hacettepe University, Ankara 06800, Turkey. Her research interests include software engineering, information systems, and Industry 4.0. Gökalp received her Ph.D. degree in information systems from Middle East Technical University. Contact her at ebrugokalp@hacettepe.edu.tr.

AHMET DİKİCİ is a senior chief team leader at ASELSAN Inc., Ankara 06200, Turkey. His research interests include software engineering, business process management, and software process improvement. Dikici received his Ph.D. degree in information systems from Middle East Technical University. Contact him at ahmetdikici@aselsan.com.

Our thorough investigation into the role of generative AI in SE uncovered a transformative technological shift that holds the potential to fundamentally reshape the development processes outlined in SWEBOK v4.[8] By examining the application of generative AI across essential SE processes, from requirements gathering to maintenance, we have identified significant opportunities to improve productivity, efficiency, and creativity. On the other hand, the success of generative AI is a function of both the quality of the underlying AI models and how effectively users can leverage those models. For instance, even the most state-of-the-art model might still produce poor results if not leveraged properly, while even a moderately advanced model could perform extremely well if placed in the hands of an expert.

The key to maximizing the benefit of generative AI is balance: leveraging technological innovation with human expertise. An organization needs to plan well and commit to transparency and ethical practices. Generative AI is designed to support human intelligence, not replace it; hence, strategies to enhance creativity and problem solving need to be instituted. In return, this allows organizations to maximize the benefits of generative AI, reducing the risk of potential misuse that includes misinformation and the invasion of privacy. Understanding generative AI as an enhancer of human capabilities is central to its responsible and effective integration.[19] Although there is a growing amount of research on human–AI collaboration, this study builds on initial efforts by delving deeply into a nuanced perspective of how generative AI could augment human decision making and productivity throughout the software development lifecycle. By addressing challenges, providing a clear implementation road map, and exploring future implications, we have established a practical framework that is applicable to general SE processes. However, it is crucial to recognize the needs of different industries, especially those that require high precision and sound logical reasoning, such as aerospace, medical device manufacturing, financial trading, traffic management, and scientific computing, necessitating special studies.

A primary limitation of this study is our focused examination of seven out of the thirteen SWEBOK v4[8] knowledge areas. This selective approach was driven by the need for deep, actionable analysis within reasonable study parameters, and the current technological readiness of different areas for AI integration. Future research should address the remaining six areas. Particularly promising directions include investigating how generative AI could enhance SE processes and methods, support professional practice through automated guidance, and optimize engineering economics through predictive modeling. This sequential approach will allow us to track technological readiness and practical applicability as generative AI capabilities mature. Future studies also need to investigate not only the current role and limitations of generative AI within such specialized contexts but also how such these roles might evolve as the technology matures. For instance, in aerospace engineering, generative AI might initially serve as a

documentation assistant before evolving into a more sophisticated verification tool; in web development, it might progress from code completion to full-stack architectural recommendations. Understanding these evolutionary pathways and domain-specific adaptations will be key to the effective integration of generative AI across diverse contexts in SE.

As we look to the future, generative AI is poised to transform software development. As AI technology advances, its capacity to automate complex tasks, enhance decision making, and drive innovation will continue to expand. To tackle this situation, software professionals must cultivate agility and adaptability. These advances are not a threat but a powerful tool for continuous improvement. The future of SE will be characterized by a collaborative relationship between human creativity and AI to lead and innovate in an increasingly digital world.

REFERENCES

1. D. Smit and H. Smuts, "The impact of GitHub Copilot on developer productivity from a software engineering body of knowledge perspective," in *Proc. Americas Conf. Inf. Syst.*, 2024, pp. 1–10.
2. M. Fu and C. Tantithamthavorn, "GPT2SP: A transformer-based agile story point estimation approach," *IEEE Trans. Softw. Eng.*, vol. 49, no. 2, pp. 611–625, Feb. 2023, doi: 10.1109/TSE.2022.3158252.
3. R. Seroter, "Finding flow: Generative AI's impact on developer productivity." Google Cloud. Accessed: 2024. [Online]. Available: URL link: https://cloud.google.com/resources/gen-ai-impact-on-dev-productivity
4. A. Rajbhoj, A. Somase, P. Kulkarni, and V. Kulkarni, "Accelerating software development using generative AI: ChatGPT case study," in *Proc. 17th Innov. Softw. Eng. Conf. (ISEC)*, New York, NY, USA: ACM, pp. 1–11, 2024, doi: 10.1145/3641399.3641403.
5. Y. Huang et al., "Generative software engineering," 2024, *arXiv:2403.02583*.
6. J. Sauvola, S. Tarkoma, M. Klemettinen, J. Riekki, and D. Doermann, "Future of software development with generative AI," *Automated Softw. Eng.*, vol. 31, no. 1, 2024, Art. no. 26, doi: 10.1007/s10515-024-00426-z.
7. R. Khojah, M. Mohamad, P. Leitner, F. G. Neto, and O. de, "Beyond code generation: An observational study of ChatGPT usage in software engineering practice," 2024, *arXiv:2404.14901*.
8. H. Washizaki, "Guide to the software engineering body of knowledge (SWEBOK guide), Version 4.0." IEEE Computer Society, Waseda Univ., Tokyo, Japan, 2024. [Online]. Available: http://www.swebok.org
9. A. Yalçıner, A. Dikici, and E, Gökalp, "Data-driven software engineering: A systematic literature review," in *Systems, Software and Services Process Improvement*, M. Yilmaz, P. Clarke, A. Riel, R. Messnarz, C. Greiner, and T. Peisl, Cham, Switzerland: Springer-Verlag, 2024, pp. 19–32, doi: 10.1007/978-3-031-71139-8.
10. X. Hou et al., "Large language models for software engineering: A systematic literature review," *ACM Trans. Software Eng. Method.*, vol. 33, no. 8, pp. 1–79, 2024, doi: 10.1145/3695988.
11. E. Naiburg, "AI as a scrum team member," *Scrum.org*, Jul. 10, 2024. Accessed: Nov. 19, 2024. [Online]. Available: https://www.scrum.org/resources/blog/ai-scrum-team-member
12. J. W. Beard et al., "Agile development: The promise, the reality, the opportunity," in *Proc. Agil-ISE@CAiSE*, 2024, pp. 7–17.
13. M. Mahboob, M. Rayyan Uddin Ahmed, Z. Zia, M. Shakeel Ali, and A. Khaleel Ahmed, "Future of artificial intelligence in agile software development," 2024, *arXiv:2408.00703*.
14. U. K. Durrani, M. Akpinar, M. F. Adak, A. T. Kabakus, M. M. Ozturk, and M. Saleh, "A decade of progress: A systematic literature review on the integration of AI in software engineering phases and activities (2013-2023)," *IEEE Access*, vol. 12, pp. 171,185–171,204, 2024, doi: 10.1109/ACCESS.2024.3488904.
15. Y. Mota, "Generative AI implementation: Comprehensive guide," Oct. 15, 2024. Accessed: Nov. 19, 2024. [Online]. Available: https://www.n-ix.com/generative-ai-implementation/
16. K. Wach et al., "The dark side of generative artificial intelligence: A critical analysis of controversies and risks of ChatGPT," *Entrepreneurial Bus. Econ. Rev.*, vol. 11, no. 2, pp. 7–30, 2023, doi: 10.15678/EBER.2023.110201.
17. R. Jha, "Future of software development with generative AI & machine learning," *Autom. Softw. Eng.*, vol. 31, no. 1, Sep. 2024, Art.no. 26.
18. P. Kumari, "7 top generative AI use case in software engineering," *Labellerr*, Feb. 12, 2024. Accessed: Nov. 19, 2024. [Online]. Available: https://www.labellerr.com/blog/the-most-elaborative-guide-for-generative-ai-with-top-7-use-cases/#:~:text=Generative%20AI%20has%20diverse%20applications,%2C%20summarization%2C%20and%20dialogue%20generation
19. C. Ebert and P. Louridas, "Generative AI for software practitioners," *IEEE Softw.*, vol. 40, no. 4, pp. 30–38, Aug. 2023, doi: 10.1109/MS.2023.3265877.